AES Constant-Time Implementation Performance Report

Assignment 3

Implementations: O0 (baseline) through O5 (final)
Reference: OpenSSL `AES_encrypt`
Profiler: Intel PIN
Goal: Table-free, secret-independent memory access

Group Member Name	Roll Number
Denchanadula Rushikesh Chary	220336
Raj Vinayak Meena	220861
Jenil Dobariya	220385
Prem Kansagra	220816
Poojal Katiyar	220770

Introduction

Standard AES implementations (including OpenSSL's `AES_encrypt`) use pre-computed S-box and T-box lookup tables indexed by secret key/state bytes. These table accesses introduce *secret-dependent memory access patterns*, making the implementation vulnerable to cache-timing side-channel attacks.

This report documents a series of six progressively optimised implementations (O0–O5) of a constant-time AES-128 encryption function. Each version eliminates all secret-dependent memory accesses by computing the S-box algebraically at runtime using Galois Field arithmetic and Fermat's Little Theorem for inversion ($a^{254} \bmod p(x)$). The reference baseline is OpenSSL's `AES_encrypt`.

Performance is measured using Intel PIN instrumentation across three key metrics:

- **Total Cycles** — the primary performance metric.
- **Total Instructions** — instruction count indicating compute efficiency.
- **Memory Operations** — data memory reads and writes.
- **Average CPI** — cycles per instruction, indicating pipeline efficiency.

Note: The OpenSSL implementation uses lookup tables and is *not* constant-time. It is provided only as a performance reference.

Performance Summary

Table 1: PIN Performance Metrics for All Implementations

Version	Total Cycles	Total Instructions	Memory Ops	Avg CPI
O0 (Baseline)	1,819,852	729,012	272,710	2.496
O1	1,620,009	650,085	242,481	2.492
O2	1,407,181	593,289	203,473	2.372
O3	1,405,742	592,914	203,207	2.371
O4 † (GCC flags, *illegal*)	1,220,001	524,657	173,836	2.325
O5 (Final Submission)	**1,307,562**	**551,974**	**188,897**	**2.369**
OpenSSL (reference)	1,200,482	503,422	174,265	2.385

† O4 uses compiler-level GCC optimisation flags (`#pragma GCC optimize("O3", "unroll-loops", ...)`), which is **not permitted** for submission. Shown for reference only.

Cycle Reduction vs. Baseline (O0)

Performance Charts

Table 2: Improvement relative to O0 (baseline)

Version	Cycles Saved vs. O0	Reduction (%)
O1	199,843	10.98%
O2	412,671	22.68%
O3	414,110	22.76%
O4 (illegal)	599,851	32.96%
O5 (final)	**512,290**	**28.15%**

Implementation Details

O0 — Baseline (Byte-by-Byte, Scalar)

Strategy: Straightforward translation of the AES-128 specification in scalar C code. All operations work on individual `uint8_t` bytes.

Key design choices:

- GF(2^8) multiplication (`gmul`) implemented as an unrolled 8-step loop with branchless conditional using `(a >> 7) * 0x1B`.
- GF inversion via Fermat's Little Theorem: a^{254} computed using 11 multiplications.
- S-box computed via algebraic affine transformation, one byte at a time.
- State stored as a `uint8_t state[16]` array in memory.
- Round key added byte-by-byte by casting `unsigned int*` to `uint8_t*`.

Metric	Value
Total Cycles	1,819,852
Total Instructions	729,012
Memory Operations	272,710
Average CPI	2.496

Bottleneck: Heavy memory traffic — all 16 state bytes live in an array and are read/written repeatedly. Scalar byte operations cannot exploit parallelism.

O1 — 32-bit Registers + Branchless Masks

Strategy: Lift state from memory into four 32-bit CPU registers (`c0`–`c3`). Eliminate multiplication-based masks in favour of zero-subtraction bitmask idiom.

Key optimisations over O0:

- Replaced `(bit) * 0xFF` masking with `(0 - (bit)) & 0xFF...` which generates single-cycle `NEG`/`AND` pairs on x86 instead of integer multiplications.
- State columns stored directly in four native `uint32_t` registers — reduces memory traffic significantly.

- SubBytes + ShiftRows fused in a single pass: each output column is assembled by byte-extracting from the four input columns with the correct ShiftRows offset already applied.
- The column layout on little-endian x86 aligns bit-shifts directly to ShiftRows geometry, eliminating separate transpose step.
- Only 4 `memcpy` reads (plaintext load) and 4 writes (ciphertext store) — all other state is register-resident.

Metric	O1 Value	Change vs. O0
Total Cycles	1,620,009	−199,843 (−10.98%)
Total Instructions	650,085	−78,927 (−10.83%)
Memory Operations	242,481	−30,229 (−11.10%)
Average CPI	2.492	−0.004

O2 — 32-bit SIMD-Style Parallel GF Arithmetic

Strategy: Compute GF operations on all 4 bytes of a column *simultaneously* within a single 32-bit word, treating each byte lane independently using carefully chosen masks.

Key optimisations over O1:

- **32-bit parallel `gmul_32`:** Multiplies 4 GF bytes in one 32-bit word at once. The mask `0x01010101` isolates each byte's LSB; `* 0xFF` (integer multiply) broadcasts it to a full byte mask. This processes 4 bytes per iteration step.
- **32-bit parallel `ginv_32`:** All 11 squarings and multiplications in the a^{254} inversion chain now operate on 4 bytes per instruction.
- **32-bit parallel affine (`affine_32`):** All 4 bit-rotation steps of the AES affine transformation applied to an entire column at once using byte-lane-preserving shift masks (`0xFEFEFEFE`, `0xFCFCFCFC`, etc.).
- **32-bit parallel MixColumns (`mix_columns_32`):** Zero-extraction technique using `ror32` eliminates per-byte extraction.

Metric	O2 Value	Change vs. O1
Total Cycles	1,407,181	−212,828 (−13.14%)
Total Instructions	593,289	−56,796 (−8.74%)
Memory Operations	203,473	−39,008 (−16.09%)
Average CPI	2.372	−0.120

Note: This version introduced a redundant computation in `mix_columns_32` — the expression `(n ^ ror32(n, 8))` was evaluated twice.

O3 — Redundancy Fix + Full Loop Unrolling with Macros

Strategy: Eliminate a Common Sub-expression in MixColumns and unroll all 10 AES rounds using C macros to eliminate loop branch overhead and improve compiler scheduling opportunities.

Key optimisations over O2:

- **Redundancy fix in MixColumns:** The expression `u = n ^ ror32(n, 8)` is now computed *once* and reused, eliminating a redundant rotate and XOR per call.
- **Full round unrolling via `FULL_ROUND` and `FINAL_ROUND` macros:** All 9 middle rounds and the final round are explicitly unrolled. The compiler sees the full computation graph and can schedule independent chains of `ginv` operations across rounds.
- **Fused `SUBBYTES_SHIFTROWS` macro:** All 4 columns' `ginv`/`affine` chains are visible simultaneously in one macro, enabling optimal dependency scheduling.
- Added `__builtin_assume_aligned` hint for round key pointer alignment.
- Added `__restrict__` qualifiers on all pointer arguments.

Metric	O3 Value	Change vs. O2
Total Cycles	1,405,742	−1,439 (−0.10%)
Total Instructions	592,914	−375 (−0.06%)
Memory Operations	203,207	−266 (−0.13%)
Average CPI	2.371	−0.001

Observation: The gain over O2 is marginal (<0.1%) because the it was a single redundant operation, and the compiler had likely already performed some scheduling of the round loops. The architectural limit of the 32-bit approach is being reached.

WARNING: Not for Submission

O4 — 64-bit Parallel GF + GCC Pragma Optimisations (ILLEGAL — NOT FOR SUBMISSION)

This version is presented for performance reference only. It uses `#pragma GCC optimize("O3", "unroll-loops", "omit-frame-pointer", "inline")` at the top of the source file, which constitutes use of the compiler's own aggressive optimisation flags. This is explicitly not allowed per the assignment rules.

Strategy: Same 64-bit parallelism as O5 (below), but with compiler optimisation pragmas enabled to explore the theoretical peak performance of the approach.

Key changes from O3:

- Upgraded to **64-bit parallel GF arithmetic** (`gmul_64`, `ginv_64`, `affine_64`): processes 8 GF bytes simultaneously in one `uint64_t` register.
- State packed into two 64-bit registers (`d0`, `d1`) instead of four 32-bit ones.

- `FULL_ROUND_64` and `FINAL_ROUND_64` macros process both column-pairs simultaneously.
- **GCC pragma flags (`#pragma GCC optimize(...)`)** allow the compiler's own O3, loop unrolling, and inlining passes — this is the disallowed element.

Metric	O4 Value	Change vs. O3
Total Cycles	1,220,001	−185,741 (−13.21%)
Total Instructions	524,657	−68,257 (−11.51%)
Memory Operations	173,836	−29,371 (−14.45%)
Average CPI	2.325	−0.046

Takeaway: This result shows the *theoretical ceiling* achievable with this 64-bit approach and the compiler's full optimisation suite. O4 comes within 1.6% of OpenSSL (1,200,482 cycles) while remaining fully constant-time. It is included purely to demonstrate potential, and is **not** submitted.

Final Submission

O5 — 64-bit Parallel GF Arithmetic, No Compiler Flags (FINAL SUBMISSION)

Strategy: All the 64-bit parallelism from O4, but without the disallowed `#pragma GCC optimize` directives. This is the best legally submittable implementation.

Overall Optimizations:

- **64-bit parallel `gmul_64`:** Processes 8 GF(2^8) bytes simultaneously inside a single `uint64_t`. Masks `0x0101010101010101ULL` and `0xFEFEFEFEFEFEFEFEULL` keep bytes independent.
- **64-bit parallel `ginv_64`:** The entire 11-step a^{254} chain operates on 8 bytes per instruction instead of 4.
- **64-bit parallel affine (`affine_64`):** The 4 bit-rotation steps of the AES affine transformation span 8 bytes simultaneously.
- **64-bit parallel `mix_columns_64`:** `ror32_in_64` rotates two independent 32-bit columns inside a 64-bit word using masks, performing MixColumns on two columns per call.
- State reduced from 4×32-bit to **2×64-bit** registers (`d0`, `d1`), halving the number of round-key XOR operations.
- **SUBBYTES_SHIFTROWS_64 macro:** SubBytes and ShiftRows fused for both column-pairs in a single macro body, processing all 16 state bytes in 2 `ginv_64` + 2 `affine_64` calls.

Metric	O5 Value	Change vs. O3
Total Cycles	1,307,562	−98,180 (−6.98%)
Total Instructions	551,974	−40,940 (−6.91%)
Memory Operations	188,897	−14,310 (−7.04%)
Average CPI	2.369	−0.002

Comparison with OpenSSL Reference

OpenSSL's `AES_encrypt` uses the T-box (T-table) approach: four 1 KB lookup tables indexed by state bytes. This achieves very high throughput at the cost of secret-dependent memory accesses. It is **not** constant-time.

Table 3: Final submission (O5) vs. OpenSSL

Metric	O5 (Final)	OpenSSL
Total Cycles	1,307,562	1,200,482
Total Instructions	551,974	503,422
Memory Operations	188,897	174,265
Average CPI	2.369	2.385
Cycle overhead vs. OpenSSL	+107,080 cycles (+8.92%)	

O5 runs only **8.92% slower** than OpenSSL's table-based implementation while being fully constant-time and free of secret-dependent memory accesses. The CPI of O5 (2.369) is actually *slightly better* than OpenSSL (2.385), indicating the algebraic pipeline uses CPU execution units efficiently. The gap is entirely due to the higher instruction count resulting from computing the S-box algebraically rather than via lookup.

Optimisation Progression Summary

Key Insight: The Cost of Constant-Time

The total overhead of achieving constant-time execution in O5 versus OpenSSL is approximately 8.92% more cycles. This is a remarkably small price for full elimination of secret-dependent memory accesses. The 64-bit parallel approach allows the CPU to compute 8 GF byte operations per instruction, closely approaching the throughput of a table lookup while remaining branch- and table-free.

Conclusion

Starting from a 1,819,852-cycle baseline (O0), the optimisation series achieved a **28.15% total cycle reduction** in the final legal submission (O5, 1,307,562 cycles). The key insight driving the most significant gains was upgrading from scalar byte operations to

Table 4: Step-by-step optimisation progression

Version	Primary Optimisation	Cycles
O0	Scalar byte-by-byte baseline; branchless GF via `* 0xFF` masks	1,819,852
O1	32-bit register state; zero-subtraction masks; fused SubBytes+ShiftRows	1,620,009
O2	32-bit parallel GF arithmetic (4 bytes/op); parallel inversion and affine	1,407,181
O3	CSE fix in MixColumns; full 10-round macro unrolling	1,405,742
O4†	64-bit parallel GF (8 bytes/op) + illegal GCC pragma flags	1,220,001
O5	**64-bit parallel GF (8 bytes/op), no compiler flags (legal)**	**1,307,562**
OpenSSL	T-box lookup tables (not constant-time, reference only)	1,200,482

† Not eligible for submission.

data-parallel GF arithmetic using 32-bit (O2) and then 64-bit (O5) register-width operations, allowing multiple AES bytes to be processed per instruction without any lookup tables.

The final O5 implementation is:

- **Constant-time**: no branches or memory accesses depend on secret data.
- **Table-free**: no pre-computed S-box or T-box arrays.
- **Within 8.92% of OpenSSL** in cycle count, despite computing the S-box algebraically.